

Polyline Editor with 3D View

Human-Computer Interaction (HCI) Mid Lab Project Documentation

Project Overview

This project was developed as part of the Human-Computer Interaction (HCI) Mid Lab. During the design process, concepts from Chapter 5 of the HCI textbook were studied and applied to guide the interaction design and usability considerations of the system.

Student Information

Name: Shaheer Ahmed

Seat No: B23110006154

Deployment Link

<https://hcidlab154.vercel.app/>

Team Members

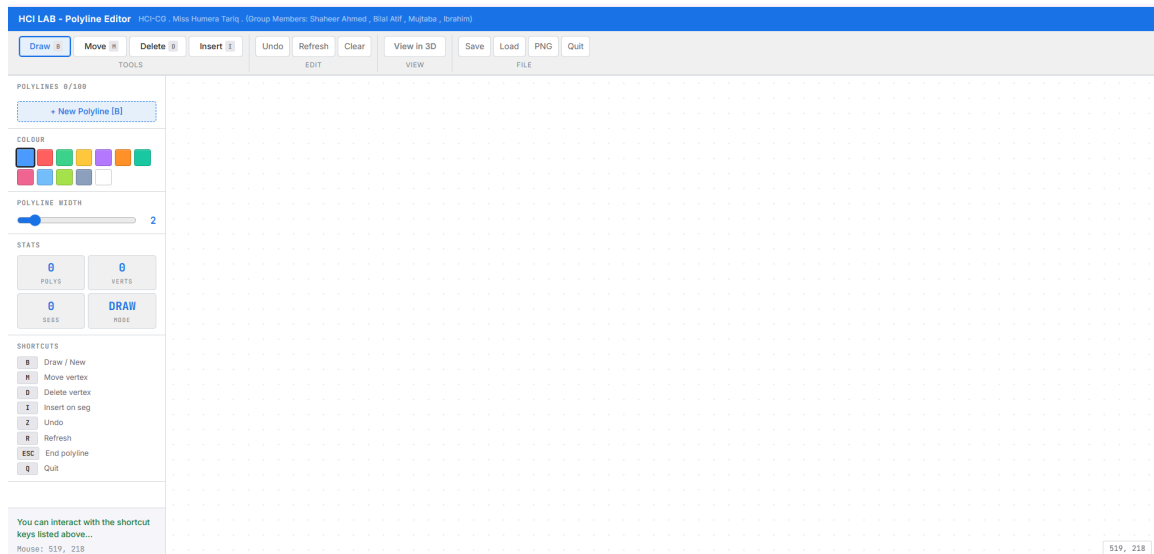
Name	Seat No	Phase
Shaheer Ahmed	B23110006154	Implementation and Deployment
M Bilal Atif Usmani	B23110006057	Analysis
Muhammad Ibrahim	B23110006103	Design
Muhammad Mujtaba	B23110006107	Requirement

My Contribution

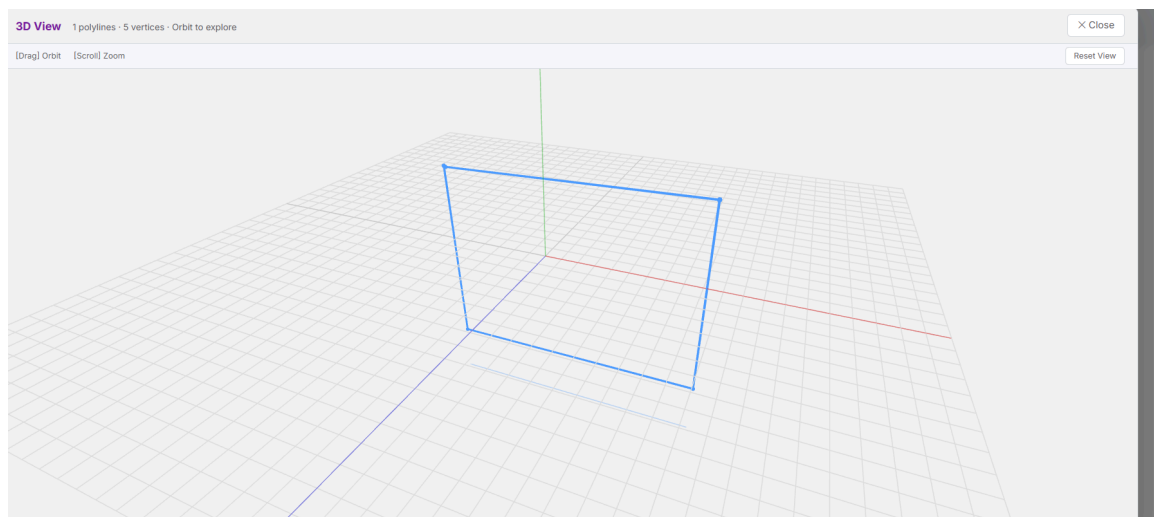
- Implemented the interactive Polyline Editor using React and TypeScript.
- Developed vertex editing features including Move, Delete, and Insert.
- Implemented the Undo history stack (maximum 30 steps).
- Created Save/Load JSON functionality for storing polylines.
- Added Export PNG functionality for saving the canvas as an image.
- Implemented 3D visualization of polylines using Three.js.
- Implemented keyboard based interaction controls.
- Designed the data structures used to store polylines and vertices.

User Interface

Editor UI Image:



3D View Image:



Technology Stack

- React.js
- Tailwind CSS
- Three.js (3D Rendering)
- HTML5 Canvas (2D Rendering)

Keyboard Shortcut Controls

Key	Action	Description
B	Begin	Start creating a new polyline
D	Delete	Remove the closest vertex
M	Move	Pick a vertex and place it at a new location
I	Insert	Insert a new point on the nearest segment
Z	Undo	Undo the last action (up to 30 steps)
R	Refresh	Redraw all polylines
ESC	End	End current polyline drawing
Q	Quit	Clear canvas and end session

Additional Functionalities

- Undo functionality using history stack
- Export canvas as PNG image
- Save and Load polylines using JSON files
- 3D visualization of polylines using Three.js
- Customizable color and stroke width

Polyline Interaction Details

Begin Mode (B)

- Click on canvas to place vertices
- Ghost preview line follows cursor
- Complete polyline via double-click, right-click, or ESC

Delete Mode (D)

- Hover near vertex to highlight
- Click to remove nearest vertex

Move Mode (M)

- Two step interaction: select vertex then place at new position

Insert Mode (I)

- Preview point appears on nearest segment
- Click inserts new vertex

3D View

- Opens Three.js viewer
- Users can rotate, zoom, and pan

Data Structures Used

The primary data structure used is a dynamic array of polyline objects. Each polyline contains an array of Point2D vertices. The undo system uses a stack (LIFO) of deep-cloned snapshots.

Type Definations

```
interface Point2D {
  x: number;
  y: number;
}

interface Polyline {
  id: number;
  name: string;
  points: Point2D[];
  color: string;
  width: number;
}

interface EditorState {
  polys: Polyline[];
  history: Polyline[][];
  nextId: number;
}
```

Deep Clone for History

```
function deepClonePolys(polys: Polyline[]): Polyline[] {  
  return polys.map(p => ({  
    ...p,  
    points: p.points.map(pt => ({ ...pt })))  
  }));  
}
```

Why Arrays Were Used (Justification for using this Data Structure)

Arrays were chosen because they provide:

- Efficient sequential traversal for rendering
- Constant time append operations
- Simple insertion and deletion using index-based operations

Operation	Complexity
Add Vertex	O(1)
Delete Vertex	O(n)
Insert Vertex	O(n)
Nearest Vertex Search	O(n)

Mechanism for Inserting a New Point

The system uses point-to-segment projection to determine the closest segment to the mouse pointer. If the distance is within a threshold (20px), a new vertex is inserted into the polyline.

The system uses point-to-segment projection. For each segment [A,B]:

$$t = \text{dot}(M - A, B - A) / \text{dot}(B - A, B - A)$$

The value of t is clamped between 0 and 1. The closest point is calculated as:

$$C = A + t(B - A)$$

If the mouse distance from C is within 20px, that segment becomes eligible for insertion.

```
points.splice(segIdx + 1, 0, closestPoint)
```

Logic Behind Saving and Reading the Array of Polylines

Save

```
const data = JSON.stringify(  
  { polys: state.polys, meta: { saved: new Date().toISOString() } },  
  null,  
  2  
);  
  
const blob = new Blob([data], { type: "application/json" });
```

Load

```
const obj = JSON.parse(text);  
  
const maxId = obj.polys.reduce(  
  (m, p) => Math.max(m, p.id),  
  0  
);  
  
dispatch({  
  type: "SET_POLYS",  
  polys: obj.polys,  
  nextId: maxId + 1  
});
```

Challenges Faced

- Implementing two-phase Move interaction
- Implementing nearest segment projection algorithm
- Handling canvas vs Three.js coordinate systems
- Preventing flicker in preview line

- Ensuring deep cloning for undo history
- Implementing orbit camera controls

Development Process

The development followed the HCI Interaction Design Lifecycle:

Requirements → Analysis → Design → Implementation

Requirements

Requirements gathering was completed in Phase 1 by **Mujtaba**.

Analysis

Phase 2 by **Bilal**. During the analysis phase, Chapter 5 of the HCI textbook was studied and considered while planning the interaction design.

The concepts from this chapter helped in understanding user tasks, interaction flow, and usability considerations for the polyline editor.

Design

This phase was completed by **Ibrahim**. During the design phase, Chapter 5 of the HCI textbook was studied and considered while planning. Several HCI principles were applied in the design, including:

- Direct manipulation
- Immediate visual feedback
- Consistent keyboard shortcuts
- Error prevention

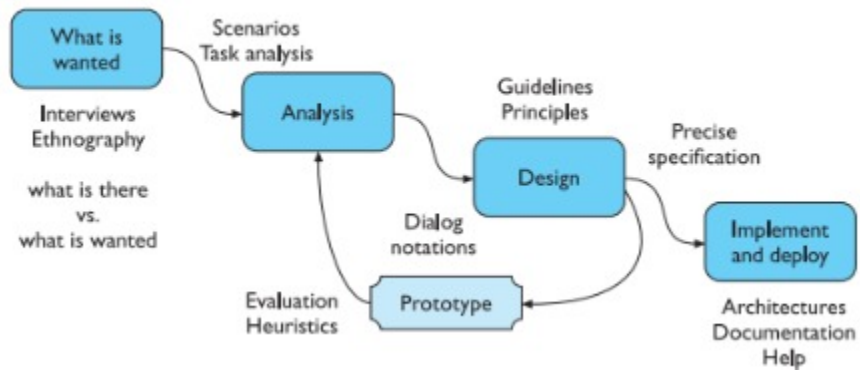
Implementation & Deployment

This phase was completed by **Shaheer Ahmed**.

The final system integrates all the required interaction features and also includes several additional improvements. The implementation focused on:

- Proper use of data structures for managing polylines and vertices
- Implementing the segment insertion mechanism inside a polyline
- Supporting keyboard shortcut-based interaction for efficient editing
- Implementing undo functionality using a history stack

- Adding a 3D viewing experience using Three.js for visualizing polylines in a layered 3D scene



References

Dix, Alan et al. Human-Computer Interaction (3rd Edition). Chapter 5 – Interaction Design Basics.